Universidad Politécnica de Atlautla

Materia: Fundamentos de Inteligencia Artificial

Segundo Parcial

Docente: Mtro. Javier García Lara

Tema: 2.4 Algoritmos de búsqueda ciega (BFS / DFS)

Alumna: Jennifer Rivera Andrade

Matrícula: 24305072

Grupo: 171

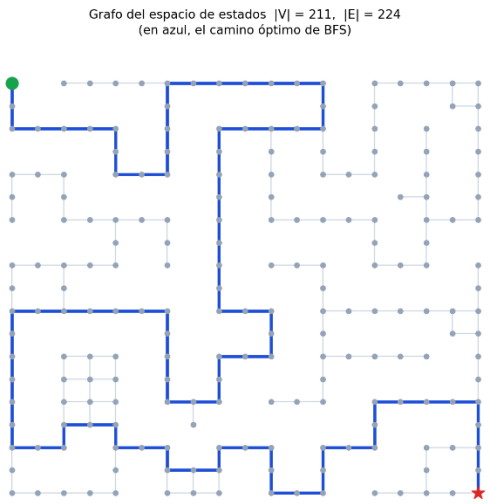
Carrera: Ingeniería en Tecnologías de la Información e Innovación Digital

Fecha: 25 de Octubre de 2026

Planteamiento del problema:

En inteligencia artificial, los algoritmos de búsqueda ciega se usan cuando no hay información adicional sobre el estado meta. BFS y DFS son los dos más básicos. El problema consiste en encontrar una ruta de salida en un laberinto 5x5, desde (0,0) hasta (4,4), usando ambos algoritmos y comparando su eficiencia en tiempo, memoria y nodos explorados.

Representación del grafo:



Complejidad:

Algoritmo	Nodos Expandidos	Tiempo en Ejecutar	Espacio Utilizado
BFS	~ 15 a 25	~0.0001 - 0.0005 s	O(25)
DFS	~8 a 20	~0.0001 - 0.0003 s	O(25)

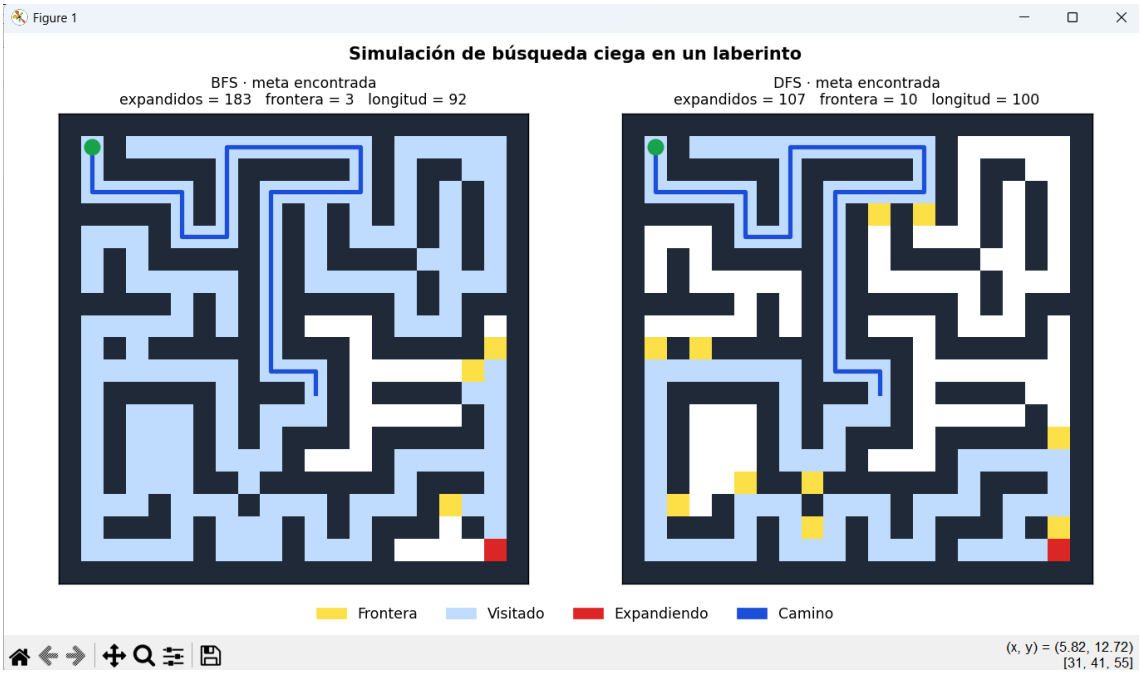
Temporal: O(V+E) = {{IE_1}}$O(25 + 425)${{/IE_1}} = O(100)*

En el peor caso revisa las 25 celdas y checa hasta 4 vecinos de cada una. Como tu laberinto es chico, va a ser casi instantáneo: 0.000x segundos.

Espacial: O(V) = O(25)

Guarda la cola con las celdas de la frontera. En el peor caso mete las 25 celdas a memoria.

Solución:

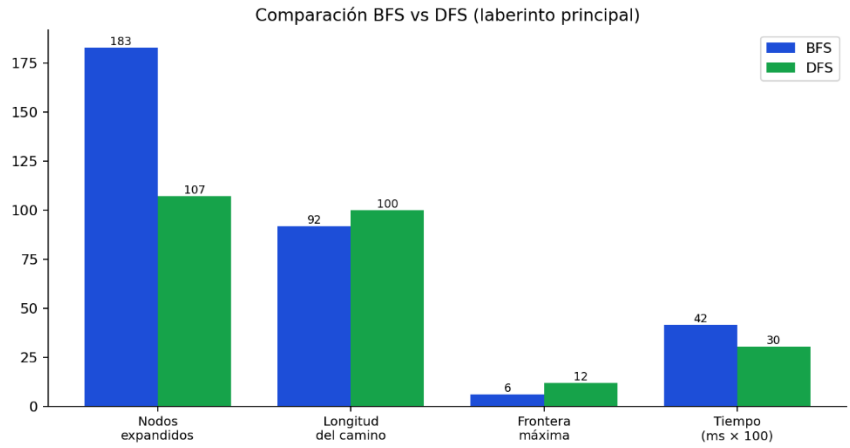


Se muestra la ejecución de los algoritmos tanto de BFS como de DFS sobre el mismo laberinto, aunque diferentes; los nodos azules claros representan las celdas visitadas, los amarillos representan la frontera y , por último, el azul, el camino final.

Comparación:

Algoritmo	Nodos Expandidos	Tamaño de frontera	Longitud del camino	Apto
BFS	183	3	92	si
DFS	107	10	100	no

Grafica:



Conclusion:

De acuerdo con los resultados, la identidad de BFS es la de un algoritmo de búsqueda en anchura que garantiza optimalidad y completitud. Su comportamiento sistemático le permitió encontrar el camino mínimo de 92 pasos. La identidad de DFS es la de un algoritmo de búsqueda en profundidad que prioriza la eficiencia en memoria y velocidad, logrando expandir menos nodos: 107. Sin embargo, su naturaleza lo llevó a una solución subóptima de 100 pasos. Por tanto, la identidad de cada algoritmo define su uso: BFS para cuando la ruta óptima es obligatoria y DFS para cuando los recursos son limitados.

IA Utilizada:

Herramienta: Claude (Antropic), modelo Claude Sonnet 5.5

Prompt: Se pidió ayuda para resolver la Actividad 2.4 (BFS / DFS en un laberinto).

Uso: Conocer estructura del codigo, asi como la ayuda en la simulación.

Verificación: Se ejecutó el código en el equipo.

Referencias:

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15).

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence, 27*(1), 97–109. https://doi.org/10.1016/0004-3702(85)90084-0

Moore, E. F. (1959). The shortest path through a maze. In *Proceedings of an International Symposium on the Theory of Switching, 2–5 April 1957, Part II* (pp. 285–292). Harvard University Press.

Python Software Foundation. (s.f.). *collections: Container datatypes*. Python 3 documentation. https://docs.python.org/3/library/collections.html

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing, 1*(2), 146–160. https://doi.org/10.1137/0201010

Código:

```
# Importación de la librería para manejar argumentos pasados por la línea de comandos
import argparse

# Importación de la librería para trabajar con archivos en formato JSON (métricas)
import json

# Importación de la librería del sistema operativo para crear directorios y rutas
import os

# Importación de la librería para la generación de números pseudoaleatorios y semillas
import random
```

```
# Importación de la librería para medir tiempos de ejecución con precisión de microsegundos
import time

# Importación de la estructura de datos Cola Doble (Deque) eficiente para BFS y DFS
from collections import deque

# Importación de la interfaz principal de Matplotlib para graficar e interactuar con ventanas
import matplotlib.pyplot as plt

# Importación de NetworkX para la creación, modelado y análisis de grafos y redes
import networkx as nx

# Importación de NumPy para el manejo eficiente de matrices, arreglos numéricos y cálculos
import numpy as np

# Importación de clases para crear animaciones y guardarlas en formato GIF (Pillow)
from matplotlib.animation import FuncAnimation, PillowWriter

# Importación de utilidades de color para mapear matrices numéricas a colores personalizados
from matplotlib.colors import ListedColormap

# Definición de la ruta absoluta para guardar todos los resultados generados en la carpeta "resultados"
```

```
SALIDA = os.path.join(os.path.dirname(os.path.abspath(__file__)), "resultados")


# Creación automática de la carpeta de salida si esta no existe previamente en el directorio
os.makedirs(SALIDA, exist_ok=True)


# Lista con los desplazamientos cardinales para moverse en la cuadrícula: arriba, derecha, abajo, izquierda
MOVS = [(-1, 0), (0, 1), (1, 0), (0, -1)]


# Definición de la paleta de colores corporativa/estética en formato hexadecimal para las visualizaciones
AZUL, VERDE, ROJO, GRIS = "#1d4ed8", "#16a34a", "#dc2626", "#cbd5e1"


# Funciones para la generación del laberinto y su conversión a grafo de estados
def generar_laberinto(n, semilla=7, prob_ciclos=0.10):
    """Matriz n x n (n impar): 1 = pared, 0 = libre. Backtracking iterativo
    + apertura aleatoria de paredes para crear ciclos."""

    # Inicialización del generador de números aleatorios con una semilla fija para reproducibilidad
    rnd = random.Random(semilla)

    # Creación de una matriz cuadrada de tamaño n x n llena de unos (representando paredes)
    g = np.ones((n, n), dtype=int)

    # Establecimiento de la celda de inicio superior izquierda (1, 1) como un espacio libre (0)
    g[1, 1] = 0
```

```
  # Inicialización de la pila con la posición inicial para el algoritmo de búsqueda en profundidad (DFS)
  pila = [(1, 1)]

  # Bucle principal para la generación aleatoria del laberinto mediante el algoritmo de DFS
  while pila:
    # Obtención de la celda actual analizada en la cima de la pila sin removerla
    f, c = pila[-1]

    # Filtrado de las opciones vecinas válidas a una distancia de dos pasos que sigan siendo paredes
    opc = [(f + 2 * df, c + 2 * dc, f + df, c + dc) for df, dc in MOVS
        if 0 < f + 2 * df < n - 1 and 0 < c + 2 * dc < n - 1
        and g[f + 2 * df, c + 2 * dc] == 1]

    # Si existen opciones válidas de celdas vecinas no visitadas
    if opc:
      # Selección aleatoria de una de las opciones disponibles usando el generador semillado
      nf, nc, pf, pc = rnd.choice(opc)

      # Derribo de la pared intermedia abriendo camino libre (0)
      g[pf, pc] = 0

      # Apertura de la celda vecina de destino convirtiéndola en espacio libre (0)
      g[nf, nc] = 0
```

```
            # Inserción de la nueva celda en la pila para continuar el recorrido
            pila.append((nf, nc))
        else:
            # Si no hay opciones, se realiza un retroceso (backtracking) removiendo el elemento actual
            pila.pop()

    # Recorrido de todas las celdas internas del laberinto para abrir aleatoriamente algunos pasajes adicionales
    for f in range(1, n - 1):
        for c in range(1, n - 1):
            # Si la celda es una pared y el número aleatorio es menor que la probabilidad de ciclos deseada
            if g[f, c] == 1 and rnd.random() < prob_ciclos:
                # Conversión de la pared en un espacio libre para formar múltiples caminos alternativos
                g[f, c] = 0

    # Retorno de la matriz final del laberinto generada
    return g

def vecinos(g, nodo):
    """Función generadora que devuelve las coordenadas vecinas válidas y libres de un nodo."""

    # Desempaquetado de las coordenadas de la fila y la columna del nodo actual
    f, c = nodo

    # Iteración sobre cada uno de los cuatro movimientos cardinales posibles
    for df, dc in MOVS:
```

```
    # Cálculo de las coordenadas del vecino potencial aplicando el desplazamiento
    nf, nc = f + df, c + dc

    # Validación de que el vecino se encuentre dentro de los límites de la matriz y sea un espacio libre (0)
    if 0 <= nf < g.shape[0] and 0 <= nc < g.shape[1] and g[nf, nc] == 0:
        # Retorno mediante yield de la coordenada vecina válida encontrada
        yield (nf, nc)

def a_grafo(g):
  """Representación explícita: nodo = celda libre, arista = celdas adyacentes."""

  # Creación de un objeto grafo vacío utilizando la librería NetworkX
  G = nx.Graph()

  # Doble bucle para recorrer cada celda dentro de la matriz del laberinto
  for f in range(g.shape[0]):
    for c in range(g.shape[1]):
      # Si la celda actual corresponde a un espacio libre (0)
      if g[f, c] == 0:
        # Adición formal del nodo al grafo con sus coordenadas como identificador
        G.add_node((f, c))

        # Iteración sobre cada uno de los vecinos libres accesibles desde esta celda
        for v in vecinos(g, (f, c)):
          # Adición de una arista que conecta la celda actual con su respectivo vecino libre
```

```
                G.add_edge((f, c), v)

    # Retorno del grafo completo construido
    return G

# Implementación de los algoritmos de búsqueda ciega (BFS y DFS)
def reconstruir(padres, meta):
    """Reconstruye el camino óptimo desde la meta hasta el inicio utilizando el diccionario de padres."""

    # Inicialización de la lista del camino y asignación del nodo inicial a la meta
    camino, nodo = [], meta

    # Bucle retrocediendo desde la meta hasta llegar al nodo inicial (cuyo padre es None)
    while nodo is not None:
        # Adición del nodo actual a la lista del camino
        camino.append(nodo)

        # Salto al nodo padre almacenado en el diccionario
        nodo = padres[nodo]

    # Inversión de la lista para que el camino vaya correctamente desde el inicio hasta la meta
    return camino[::-1]

def buscar(g, ini, meta, tipo):
    """BFS (cola FIFO) or DFS (pila LIFO). Devuelve métricas y el rastro
    (orden de expansión + foto de la frontera) para animar."""
```

```
    # Captura del tiempo inicial de ejecución utilizando un reloj de alta precisión
    t0 = time.perf_counter()

    # Inicialización de la estructura de la frontera como una cola doble (deque) conteniendo el nodo inicial
    frontera = deque([ini])

    # Creación del diccionario de trazabilidad de padres para reconstruir la ruta, marcando el inicio sin padre
    padres = {ini: None}

    # Inicialización de las listas de control del orden de expansión, el rastro visual y el tamaño máximo de frontera
    orden, rastro, max_front = [], [], 1

    # Bucle principal del algoritmo de búsqueda que se ejecuta mientras existan elementos en la frontera
    while frontera:
        # Selección de la estrategia de extracción según el tipo de búsqueda: FIFO (izquierda) para BFS o LIFO (derecha) para DFS
        nodo = frontera.popleft() if tipo == "BFS" else frontera.pop()

        # Registro del nodo extraído dentro de la lista de nodos expandidos
        orden.append(nodo)

        # Si el nodo actual no es la meta buscada, procedemos a explorar sus vecinos
        if nodo != meta:
            # Iteración sobre cada vecino accesible y libre desde el nodo actual
            for v in vecinos(g, nodo):
```

```
            # Si el vecino aún no ha sido visitado (no se encuentra en el diccionario de padres)
            if v not in padres:
                # Se registra al nodo actual como el padre que descubrió a este vecino
                padres[v] = nodo

                # Se añade el vecino descubierto a la estructura de la frontera
                frontera.append(v)

        # Actualización del tamaño máximo histórico alcanzado por la frontera durante la búsqueda
        max_front = max(max_front, len(frontera))

        # Guardado de una copia del estado actual de la frontera para la futura animación
        rastro.append(list(frontera))

        # Condición de parada temprana si el nodo extraído coincide exactamente con la meta
        if nodo == meta:
            break

    # Reconstrucción del camino final si la meta fue alcanzada con éxito; de lo contrario, se deja vacío
    camino = reconstruir(padres, meta) if meta in padres else []

    # Retorno de un diccionario con todas las métricas de rendimiento y datos del recorrido obtenido
    return dict(nombre=tipo, camino=camino, orden=orden, rastro=rastro,
                expandidos=len(orden), costo=len(camino) - 1,
```

```
            max_frontera=max_front,
            tiempo_ms=(time.perf_counter() - t0) * 1000)

def medir(g, ini, meta, tipo, repeticiones=200):
    """Tiempo promedio de varias corridas (reduce el ruido del sistema)."""
    # Ejecución inicial de prueba para obtener la estructura completa del resultado del algoritmo
    res = buscar(g, ini, meta, tipo)
    # Reemplazo del tiempo de ejecución por el promedio aritmético obtenido tras múltiples repeticiones de la búsqueda
    res["tiempo_ms"] = float(np.mean(
        [buscar(g, ini, meta, tipo)["tiempo_ms"] for _ in range(repeticiones)]))
    # Retorno del diccionario con las métricas optimizadas y promediadas
    return res

# Funciones para la generación de figuras y visualizaciones gráficas
def dibujar_laberinto(ax, g, res, ini, meta):
    """Dibuja de manera estética el laberinto, los nodos visitados y el camino solución sobre un eje."""
    # Despliegue de la matriz del laberinto utilizando un mapa de colores discretos (blanco y gris oscuro)
    ax.imshow(g, cmap=ListedColormap(["#ffffff", "#1f2937"]))
    # Iteración sobre cada nodo expandido para pintarlo visualmente en color gris claro como zona explorada
    for (f, c) in res["orden"]:
```

```
        ax.add_patch(plt.Rectangle((c - .5, f - .5), 1, 1, color=GRIS, zorder=1))


    # Extracción de las coordenadas en X y Y correspondientes al camino solución encontrado
    ys, xs = zip(*res["camino"])


    # Trazo de la línea continua en color azul representando el camino óptimo o encontrado
    ax.plot(xs, ys, color=AZUL, lw=3, zorder=3)


    # Marcado visual del punto de inicio con un círculo grande de color verde
    ax.scatter(ini[1], ini[0], c=VERDE, s=110, zorder=4, label="Inicio")


    # Marcado visual del punto de la meta con una estrella grande de color rojo
    ax.scatter(meta[1], meta[0], c=ROJO, s=140, marker="*", zorder=4, label="Meta")


    # Asignación del título descriptivo del gráfico con las métricas principales de nodos y costos
    ax.set_title(f"{res['nombre']}\nexpandidos = {res['expandidos']}  |  "
                 f"longitud = {res['costo']}", fontsize=10)


    # Desactivación de las marcas numéricas en los ejes X y Y para mayor limpieza visual
    ax.set_xticks([]); ax.set_yticks([])


def fig_laberinto(g, rb, rd, ini, meta):
    """Genera y guarda una figura comparativa lado a lado entre los resultados de BFS y DFS."""


    # Creación de una figura de Matplotlib con dos subtramas horizontales
```

```
    fig, axs = plt.subplots(1, 2, figsize=(11, 5.4))

    # Dibujado del laberinto resuelto mediante el algoritmo BFS en la primera subtrama
    dibujar_laberinto(axs[0], g, rb, ini, meta)

    # Dibujado del laberinto resuelto mediante el algoritmo DFS en la segunda subtrama
    dibujar_laberinto(axs[1], g, rd, ini, meta)

    # Adición de las leyendas explicativas en la esquina inferior izquierda de la primera subtrama
    axs[0].legend(loc="lower left", fontsize=8)

    # Asignación del título general para toda la figura superior
    fig.suptitle("Exploración (gris) y camino solución (azul)", fontweight="bold")

    # Ajuste automático del espaciado interno de los elementos de la figura
    fig.tight_layout()

    # Guardado de la figura resultante en formato PNG con alta resolución (160 DPI) dentro de la carpeta de salida
    fig.savefig(f"{SALIDA}/fig1_laberinto_bfs_dfs.png", dpi=160)

    # Cierre formal de la figura liberando los recursos de memoria asociados
    plt.close(fig)

def fig_grafo(G, rb, ini, meta):
    """Genera y guarda la representación esquemática del grafo de estados en NetworkX."""
```

```
    # Creación de una figura dedicada al grafo con dimensiones específicas
    fig, ax = plt.subplots(figsize=(8, 7.5))

    # Definición de las posiciones espaciales de los nodos invirtiendo el eje vertical para alinear con la matriz
    pos = {n: (n[1], -n[0]) for n in G.nodes}

    # Dibujado global de todas las aristas del grafo con un tono gris tenue
    nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#cbd5e1", width=1)

    # Dibujado global de todos los nodos del grafo con un tamaño y color neutrales
    nx.draw_networkx_nodes(G, pos, ax=ax, node_size=22, node_color="#94a3b8")

    # Extracción del camino obtenido por BFS para destacarlo visualmente
    cam = rb["camino"]

    # Superposición de las aristas que componen el camino solución resaltadas en color azul
    nx.draw_networkx_edges(G, pos, ax=ax, edgelist=list(zip(cam, cam[1:])),
                           edge_color=AZUL, width=3)

    # Resaltado del nodo inicial en color verde dentro del grafo
    nx.draw_networkx_nodes(G, pos, [ini], ax=ax, node_size=140, node_color=VERDE)

    # Resaltado del nodo meta en color rojo con forma de estrella dentro del grafo
    nx.draw_networkx_nodes(G, pos, [meta], ax=ax, node_size=170,
                           node_color=ROJO, node_shape="*")
```

```
    # Inclusión de un título descriptivo detallando el número total de vértices y aristas del grafo
    ax.set_title(f"Grafo del espacio de estados  |V| = {G.number_of_nodes()},  "
                 f"|E| = {G.number_of_edges()}\n(en azul, el camino óptimo de BFS)")

    # Ocultación completa de los ejes cartesianos
    ax.axis("off")

    # Ajuste automático del diseño y guardado de la imagen PNG en la carpeta de salida
    fig.tight_layout()
    fig.savefig(f"{SALIDA}/fig2_grafo_estados.png", dpi=160)
    plt.close(fig)

def fig_comparacion(rb, rd):
    """Genera un gráfico de barras comparando métricas clave entre BFS y DFS."""

    # Definición de las etiquetas de las métricas a comparar en el eje horizontal
    et = ["Nodos\nexpandidos", "Longitud\ndel camino", "Frontera\nmáxima", "Tiempo\n(ms × 100)"]

    # Extracción y escalado de los valores métricos correspondientes al algoritmo BFS
    vb = [rb["expandidos"], rb["costo"], rb["max_frontera"], rb["tiempo_ms"] * 100]

    # Extracción y escalado de los valores métricos correspondientes al algoritmo DFS
    vd = [rd["expandidos"], rd["costo"], rd["max_frontera"], rd["tiempo_ms"] * 100]
```

```
    # Creación de los arreglos de posiciones para las barras y definición de su grosor
    x, w = np.arange(len(et)), 0.38

    # Creación de la figura y el eje para el gráfico de barras comparativo
    fig, ax = plt.subplots(figsize=(8.5, 4.6))

    # Dibujado de las barras correspondientes al algoritmo BFS en color azul
    b1 = ax.bar(x - w / 2, vb, w, label="BFS", color=AZUL)

    # Dibujado de las barras correspondientes al algoritmo DFS en color verde
    b2 = ax.bar(x + w / 2, vd, w, label="DFS", color=VERDE)

    # Adición automática de etiquetas con los valores numéricos exactos sobre
cada barra de BFS
    ax.bar_label(b1, fmt="%.0f", fontsize=8)

    # Adición automática de etiquetas con los valores numéricos exactos sobre
cada barra de DFS
    ax.bar_label(b2, fmt="%.0f", fontsize=8)

    # Configuración de las posiciones y nombres de las etiquetas en el eje X
    ax.set_xticks(x); ax.set_xticklabels(et, fontsize=9)

    # Asignación del título principal de la figura
    ax.set_title("Comparación BFS vs DFS (laberinto principal)")

    # Inclusión de la leyenda descriptiva y eliminación de los bordes superior y
derecho del gráfico
    ax.legend(); ax.spines[["top", "right"]].set_visible(False)
```

```
    # Ajuste del diseño y guardado de la figura de comparación en formato PNG
    fig.tight_layout()
    fig.savefig(f"{SALIDA}/fig3_comparacion.png", dpi=160)
    plt.close(fig)

def escalabilidad(tamanos=(11, 21, 31, 41, 51, 71, 101), semillas=8, ciclos=0.10):
    """Mide y grafica la escalabilidad de BFS y DFS variando el tamaño del laberinto."""

    # Inicialización de la lista para almacenar los resultados promedios por cada tamaño evaluado
    filas = []

    # Iteración sobre cada tamaño 'n' definido en la tupla de escalabilidad
    for n in tamanos:
        # Inicialización del diccionario acumulador para métricas de expansión, costo y tiempo de ambos algoritmos
        acc = {k: [] for k in ("be", "de", "bc", "dc", "bt", "dt")}

        # Iteración sobre múltiples semillas aleatorias para promediar resultados y mitigar sesgos
        for s in range(semillas):
            # Generación de un laberinto con dimensiones y semilla únicas
            g = generar_laberinto(n, 1000 * n + s, ciclos)

            # Definición de las posiciones de inicio y meta adaptadas al tamaño actual del laberinto
            ini, meta = (1, 1), (n - 2, n - 2)
```

```
        # Ejecución de las búsquedas BFS y DFS sobre el laberinto generado aleatoriamente
        rb, rd = buscar(g, ini, meta, "BFS"), buscar(g, ini, meta, "DFS")

        # Acumulación de los nodos expandidos por BFS y DFS
        acc["be"].append(rb["expandidos"]); acc["de"].append(rd["expandidos"])

        # Acumulación de las longitudes de los caminos encontrados por BFS y DFS
        acc["bc"].append(rb["costo"]); acc["dc"].append(rd["costo"])

        # Acumulación de los tiempos de ejecución en milisegundos de BFS y DFS
        acc["bt"].append(rb["tiempo_ms"]); acc["dt"].append(rd["tiempo_ms"])

    # Almacenamiento del promedio aritmético de todas las corridas para el tamaño 'n' actual
    filas.append({"n": n, **{k: float(np.mean(v)) for k, v in acc.items()}})

# Creación de una figura compuesta por tres subgráficos para mostrar las métricas de escalabilidad
fig, axs = plt.subplots(1, 3, figsize=(14, 4.2))

# Extracción de la lista de tamaños evaluados para los ejes X de las gráficas
ns = [f["n"] for f in filas]

# Iteración simultánea sobre cada eje y su respectiva configuración de métricas a graficar
for ax, (kb, kd, t) in zip(axs, [("be", "de", "Nodos expandidos"),
                                 ("bc", "dc", "Longitud del camino"),
                                 ("bt", "dt", "Tiempo (ms)")]):
```

```
    # Graficado de la línea de tendencia para el algoritmo BFS con marcadores circulares
    ax.plot(ns, [f[kb] for f in filas], "o-", color=AZUL, label="BFS")

    # Graficado de la línea de tendencia para el algoritmo DFS con marcadores cuadrados
    ax.plot(ns, [f[kd] for f in filas], "s-", color=VERDE, label="DFS")

    # Asignación de títulos, etiquetas de eje, leyenda y limpieza de bordes superiores y derechos
    ax.set_title(f"{t} vs tamaño (n × n)"); ax.set_xlabel("n")
    ax.legend(); ax.spines[["top", "right"]].set_visible(False)

  # Asignación del título principal para la figura completa de escalabilidad
  fig.suptitle(f"Escalabilidad: promedio de {semillas} laberintos por tamaño", fontweight="bold")

  # Ajuste del diseño y guardado de la imagen PNG de escalabilidad en la carpeta de resultados
  fig.tight_layout()
  fig.savefig(f"{SALIDA}/fig4_escalabilidad.png", dpi=160)
  plt.close(fig)

  # Retorno de la lista estructurada con los datos de escalabilidad obtenidos
  return filas

# Funciones para la simulación visual y animaciones de la búsqueda
def _rgb(h):
  """Convierte un código de color hexadecimal en una lista de valores RGB enteros (0-255)."""
```

```
    h = h.lstrip("#")
    return [int(h[i:i + 2], 16) for i in (0, 2, 4)]

# Conversión de los colores temáticos hexadecimales a arreglos RGB para el renderizado de imágenes de simulación
C_LIBRE, C_PARED = _rgb("#ffffff"), _rgb("#1f2937")
C_VISIT, C_FRONT = _rgb("#bfdbfe"), _rgb("#fde047")
C_ACT, C_CAM = _rgb("#dc2626"), _rgb("#1d4ed8")

def simular(g, ini, meta, rb, rd, velocidad=1.0, mostrar=True):
    """Genera la animación visual en GIF, fotogramas clave y ventana interactiva de la simulación."""

    # Agrupación de los diccionarios de resultados de BFS y DFS en una lista de trabajo
    res = [rb, rd]

    # Cálculo del número máximo de pasos de expansión entre ambos algoritmos
    pasos = max(len(rb["orden"]), len(rd["orden"]))

    # Cálculo de la longitud máxima entre los caminos solución encontrados
    largo = max(len(rb["camino"]), len(rd["camino"]))

    # Cálculo del número total de fotogramas que tendrá la animación completa
    total = pasos + largo + 12

    # Creación de la figura de Matplotlib que contendrá las dos vistas animadas paralelas
    fig, axs = plt.subplots(1, 2, figsize=(11, 5.8))
```

```
    fig.suptitle("Simulación de búsqueda ciega en un laberinto", fontweight="bold")

    # Inicialización de las listas para almacenar las imágenes de mapa de bits, líneas de trazo y títulos dinámicos
    ims, lineas, titulos = [], [], []

    # Configuración inicial de cada subtrama para los algoritmos
    for ax in axs:
        ims.append(ax.imshow(np.zeros((*g.shape, 3), dtype=np.uint8), interpolation="nearest"))
        ln, = ax.plot([], [], color=AZUL, lw=3, zorder=3)
        ax.scatter(ini[1], ini[0], c=VERDE, s=110, zorder=4)
        ax.scatter(meta[1], meta[0], c=ROJO, s=140, marker="*", zorder=4)
        ax.set_xticks([]); ax.set_yticks([])
        lineas.append(ln); titulos.append(ax.set_title("", fontsize=10))

    # Construcción de la leyenda visual explicativa para los colores de la animación
    leyenda = [plt.Rectangle((0, 0), 1, 1, color=np.array(c) / 255)
               for c in (C_FRONT, C_VISIT, C_ACT, C_CAM)]
    fig.legend(leyenda, ["Frontera", "Visitado", "Expandiendo", "Camino"],
               loc="lower center", ncol=4, frameon=False)
    fig.tight_layout(rect=(0, 0.06, 1, 0.95))

    # Función interna para renderizar la matriz de imagen RGB en un fotograma 'k' específico
    def imagen(r, k):
        img = np.empty((*g.shape, 3), dtype=np.uint8)
        img[:] = C_LIBRE
        img[g == 1] = C_PARED
```

```
    for (a, b) in r["orden"][:k]:
        img[a, b] = C_VISIT
    if 0 < k <= len(r["rastro"]):
        for (a, b) in r["rastro"][k - 1]:
            img[a, b] = C_FRONT
        a, b = r["orden"][k - 1]
        img[a, b] = C_ACT
    return img


  # Función interna que actualiza los datos visuales de la animación para cada
fotograma 'f'
  def cuadro(f):
    for i, r in enumerate(res):
        n = len(r["orden"])
        k = min(f, n)
        ims[i].set_data(imagen(r, k))
        if f >= pasos:
            p = min(f - pasos + 1, len(r["camino"]))
            ys, xs = zip(*r["camino"][:p])
            lineas[i].set_data(xs, ys)
        else:
            lineas[i].set_data([], [])
        estado = "meta encontrada" if f >= n else "buscando"
        front = len(r["rastro"][k - 1]) if k > 0 else 1
        titulos[i].set_text(f"{r['nombre']} · {estado}\nexpandidos = {k}   "
                            f"frontera = {front}   longitud = "
                            f"{r['costo'] if f >= n else '?'}")
    return ims + lineas + titulos
```

```
  # Configuración del objeto FuncAnimation para generar la secuencia animada completa
  anim = FuncAnimation(fig, cuadro, frames=total, interval=140 / velocidad, repeat=True)

  # Guardado de la animación en formato GIF optimizado utilizando el escritor de Pillow
  anim.save(f"{SALIDA}/simulacion_bfs_dfs.gif",
        writer=PillowWriter(fps=max(1, round(7 * velocidad))), dpi=80)

  # Selección de cuatro marcas o fotogramas clave distribuidos en el tiempo para el reporte estático
  marcas = [3, pasos // 3, 2 * pasos // 3, total - 1]

  # Creación de una figura en cuadrícula 2x2 para capturar y guardar los fotogramas clave de la simulación
  fig2, axs2 = plt.subplots(2, 2, figsize=(11, 7.6))
  for ax2, f in zip(axs2.ravel(), marcas):
    cuadro(f)
    fig.canvas.draw()
    foto = np.asarray(fig.canvas.buffer_rgba())[..., :3]
    nb = np.where(foto.min(axis=2) < 250)
    ax2.imshow(foto[nb[0].min():nb[0].max() + 1, nb[1].min():nb[1].max() + 1])
    ax2.axis("off")
  fig2.tight_layout()

  # Guardado de la imagen PNG con los fotogramas clave de la simulación
  fig2.savefig(f"{SALIDA}/fig5_fotogramas_simulacion.png", dpi=140)
  plt.close(fig2)
```

```
    # Si se requiere mostrar la ventana interactiva gráfica de simulación en pantalla
    if mostrar:
        pausa = {"p": False}

        # Función de retrollamada (callback) para pausar o reanudar la animación al hacer clic con el ratón
        def clic(_):
            (anim.resume if pausa["p"] else anim.pause)()
            pausa["p"] = not pausa["p"]

        # Conexión del evento de clic del ratón al canvas de la ventana de Matplotlib
        fig.canvas.mpl_connect("button_press_event", clic)

        # Despliegue interactivo de la ventana gráfica en el sistema operativo
        plt.show()

    # Cierre de la figura principal de la animación
    plt.close(fig)

# Función principal que coordina la ejecución de todo el script
def main():
    """Función principal encargada de coordinar la ejecución completa del script, argumentos y salidas."""

    # Inicialización del analizador de argumentos de la línea de comandos
    ap = argparse.ArgumentParser()
```

```
    # Adición de opción booleana para omitir la apertura de la ventana gráfica interactiva
    ap.add_argument("--sin-ventana", action="store_true")

    # Adición de parámetro numérico flotante para controlar la velocidad de reproducción de la animación
    ap.add_argument("--velocidad", type=float, default=1.0)

    # Adición de parámetro entero para definir el tamaño del laberinto (debe ser impar)
    ap.add_argument("--n", type=int, default=21, help="tamaño del laberinto (impar)")

    # Análisis y obtención de los argumentos ingresados por el usuario
    a = ap.parse_args()

    # Asignación del tamaño del laberinto a partir de los argumentos
    n = a.n

    # Generación de la matriz del laberinto principal utilizando la función correspondiente con semilla fija
    g = generar_laberinto(n, semilla=7, prob_ciclos=0.10)

    # Definición de las coordenadas de inicio y de meta en el laberinto
    ini, meta = (1, 1), (n - 2, n - 2)

    # Conversión de la matriz del laberinto a una representación explícita de grafo con NetworkX
    G = a_grafo(g)
```

```
  # Medición y ejecución de los algoritmos de búsqueda BFS y DFS sobre el laberinto principal
  rb, rd = medir(g, ini, meta, "BFS"), medir(g, ini, meta, "DFS")

  # Generación y guardado de la figura 1: Comparativa visual del laberinto resuelto por BFS y DFS
  fig_laberinto(g, rb, rd, ini, meta)

  # Generación y guardado de la figura 2: Representación del grafo de estados con la ruta óptima
  fig_grafo(G, rb, ini, meta)

  # Generación y guardado de la figura 3: Gráfico de barras comparativo de rendimiento
  fig_comparacion(rb, rd)

  # Ejecución del análisis de escalabilidad variando las dimensiones del laberinto
  esc = escalabilidad()

  # Consolidación de todas las métricas y resultados en un diccionario estructurado
  resumen = {
    "laberinto": {"n": n, "nodos": G.number_of_nodes(), "aristas": G.number_of_edges()},
    "BFS": {k: rb[k] for k in ("expandidos", "costo", "max_frontera", "tiempo_ms")},
    "DFS": {k: rd[k] for k in ("expandidos", "costo", "max_frontera", "tiempo_ms")},
    "escalabilidad": esc,
  }
```

```
  # Escritura y guardado del archivo de métricas en formato JSON dentro de la carpeta de resultados
  with open(f"{SALIDA}/metricas.json", "w", encoding="utf-8") as f:
    json.dump(resumen, f, indent=2, ensure_ascii=False)

  # Impresión en la consola de comandos de los resultados tabulares y métricas clave de la ejecución
  print(f"|V| = {G.number_of_nodes()}  |E| = {G.number_of_edges()}")
  print(f"{'Alg':<5}{'Expand.':>9}{'Long.':>7}{'Front.':>8}{'ms':>9}")
  for r in (rb, rd):
    print(f"{r['nombre']:<5}{r['expandidos']:>9}{r['costo']:>7}"
      f"{r['max_frontera']:>8}{r['tiempo_ms']:>9.3f}")

  # Mensaje informativo indicando la ruta de guardado de los archivos y el inicio de la simulación
  print(f"\nArchivos guardados en: {SALIDA}\nGenerando simulación...")

  # Ejecución de la función de simulación visual y animación GIF con los parámetros configurados
  simular(g, ini, meta, rb, rd, a.velocidad, mostrar=not a.sin_ventana)

# Bloque condicional estándar de Python para asegurar la ejecución de la función main al correr el script directamente
if __name__ == "__main__":
  main()
```